KAOS — Interview Prep Pack

Everything you need to explain, defend, and answer questions about the **KAOS (Kafka Automated Ops System)** project in an AI-engineering interview.

What's in here

File	What it's for	Read it when...
01_ARCHITECTURE_AND_COMPONENTS.md	The full report — **how each component is built and *why***, the end-to-end architecture, diagrams, and an honest limitations section.	You want to *understand* the system deeply.
02_INTERVIEW_QUESTIONS.md	**40 questions** (30 deep + 10 quick-fire) with model answers and "go deeper" follow-up notes, grouped by theme.	You want to *rehearse* out loud.

The 30-second version (memorize this)

> KAOS is an **event-driven, closed-loop CI/CD platform**. External signals (Sentry, GitHub, deploys) enter through an **Ingestion** anti-corruption layer that normalizes them into **Kafka** events. Three autonomous agents react by lifecycle phase: a **Triager** routes bugs to the right active owner using a **Neo4j** org graph and files Jira/Notion/Slack; a **Review Manager** assigns reviewers and tracks PRs; an LLM-driven **Ops Manager** diagnoses deploy failures with **AWS Bedrock** and **re-injects them as new bug reports — closing the loop**. A **Chatbot** answers "what happened?" via RAG over the event store, and a **Control Plane** orchestrates and simulates everything.

The five things that make this project interesting (lead with these)

1. **The closed loop** — a production failure becomes a new, pre-assigned bug report and the whole lifecycle re-runs automatically. *This is the headline.*
2. **Graph-based intelligent routing** — a 3-tier fallback (active owner → active contributor → manager escalation) in Cypher, correct by construction.
3. **Hybrid execution model** — deterministic pipelines for known workflows, LLM agents for open-ended judgement (diagnosis, Q&A). *Use the LLM for judgement, code for procedure.*
4. **Hand-rolled ReAct loop** over Bedrock with hard guardrails (temp 0, 5-iteration cap, constrained toolset) — transparency and control over framework magic.
5. **Event store as memory** — SQLite event log that provides idempotency, cross-system linkage, and a queryable history the chatbot narrates.

Tech stack cheat sheet

- **Language:** Python 3.11+
- **Event bus:** Apache Kafka (Confluent Cloud), SASL_SSL
- **Graph/brain:** Neo4j Aura
- **State/memory:** SQLite via SQLAlchemy (→ Postgres-ready via `DATABASE_URL`)
- **LLM:** AWS Bedrock — Amazon Nova Lite (agent loop) + Titan Text Express (log diagnosis), via LangChain `ChatBedrock`
- **Embeddings:** sentence-transformers `all-MiniLM-L6-v2` (local)
- **Web:** FastAPI (Ingestion :8000, Chatbot :8001, Control Plane :8080) + vanilla JS UI
- **Integrations:** Jira, Notion, Slack, GitHub
- **Resilience:** exponential-backoff-with-jitter retry decorator; JSON structured logging

Interview strategy

- **Open with the closed loop.** It's the most memorable idea — lead with it, then explain the pieces that make it possible.
- **Volunteer the limitations** (§10 of the architecture report). Naming the demo simplifications before the interviewer finds them reads as senior judgement.
- **Have the "why" ready, not just the "what."** Every component answer should end with a trade-off ("I chose X over Y because…").
- **For the AI-specific role,** spend your energy on Section D of the question bank (agents, ReAct, deterministic-vs-LLM, RAG, guardrails, evals) — that's where the signal is.